

Database System- Concepts and Architecture

Unit 2

Data Abstraction

Data abstraction is the technique of hiding the complexity of the database to its users. There are three levels of data abstraction.

Physical Level or Internal Level:

- It is the lowest level of abstraction and describes how the data in the database are actually stored.
- This level describes complex low-level data structures in detail and is concerned with the way the data is physically stored.
- Data only exists at physical level.

Logical Level or Conceptual Level:

- This is the next higher level of abstraction and describes what data are stored in the database, and what relationships exist among those data.
- It describes the structure of whole database and hides details of physical storage structure. It concentrates on describing entities, data types, relationships, attributes and constraints.
- All of the views must be derivable from this conceptual schema.

View Level or External Level:

- It is the highest level of abstraction and is concerned with the way the data is seen by individual users.
- This level simplifies the users' interaction with the system. It includes a number of user views and hence is guided by the end user requirement.
- It describes only those part of the database in which the users are interested and hides rest of all from those users.
- Each user group refers to its own external schema.

Example:

view level

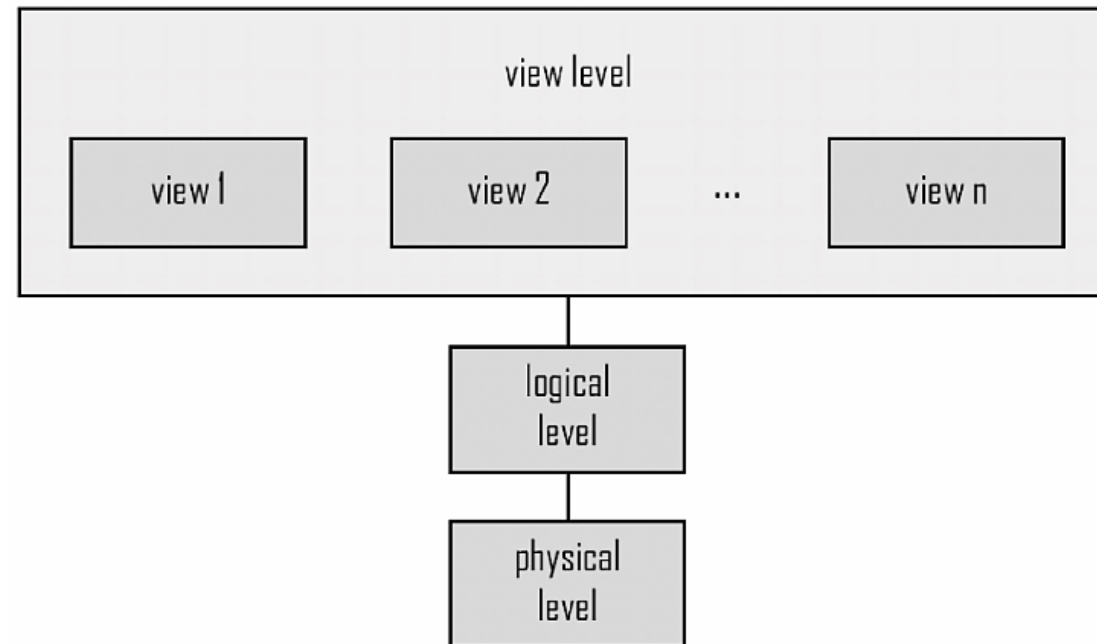
- View result
- View student information

logical level: entire database schema

- Courses (CourseNo, CourseName, Credits, Dept)
- Student (StudentID, Lname, Fname, Level, Major)
- Grade (StudentID, CourseNo, mark)

physical level:

- how these tables are stored, how many bytes it required etc.



Data Models

Data models describe the underlying structure of database. It is a conceptual tool for describing data, relationship among data, data semantics and consistency constraints.

There are several data models:.

1. Entity-relationship model
2. Object-oriented model
3. Relational model
4. Network model
5. Hierarchical Model

a. Entity Relationship Model

- E-R model describes the design of database in terms of entities and relationship among them.
- An entity is a “thing” or “object” in real world that are distinguishable from other objects. An entity is described by a set of attributes.
- For example : Attributes `account_number` and `balance` may describe entity “account”.
 - Attributes `customer_id`, `customer_name`, `customer_city` may describe entity “customer”.
- A relationship is an association among several entities. For example, a depositor relationship associates a customer with each account he or she has.
- The set of all entities of same type called entity set and similarly set of all relationship of the same type called relationship set.
- E-R model graphically express overall logical structure of a database by an E-R diagram.
- Components of E-R diagram are as follows

rectangles: represent entity sets

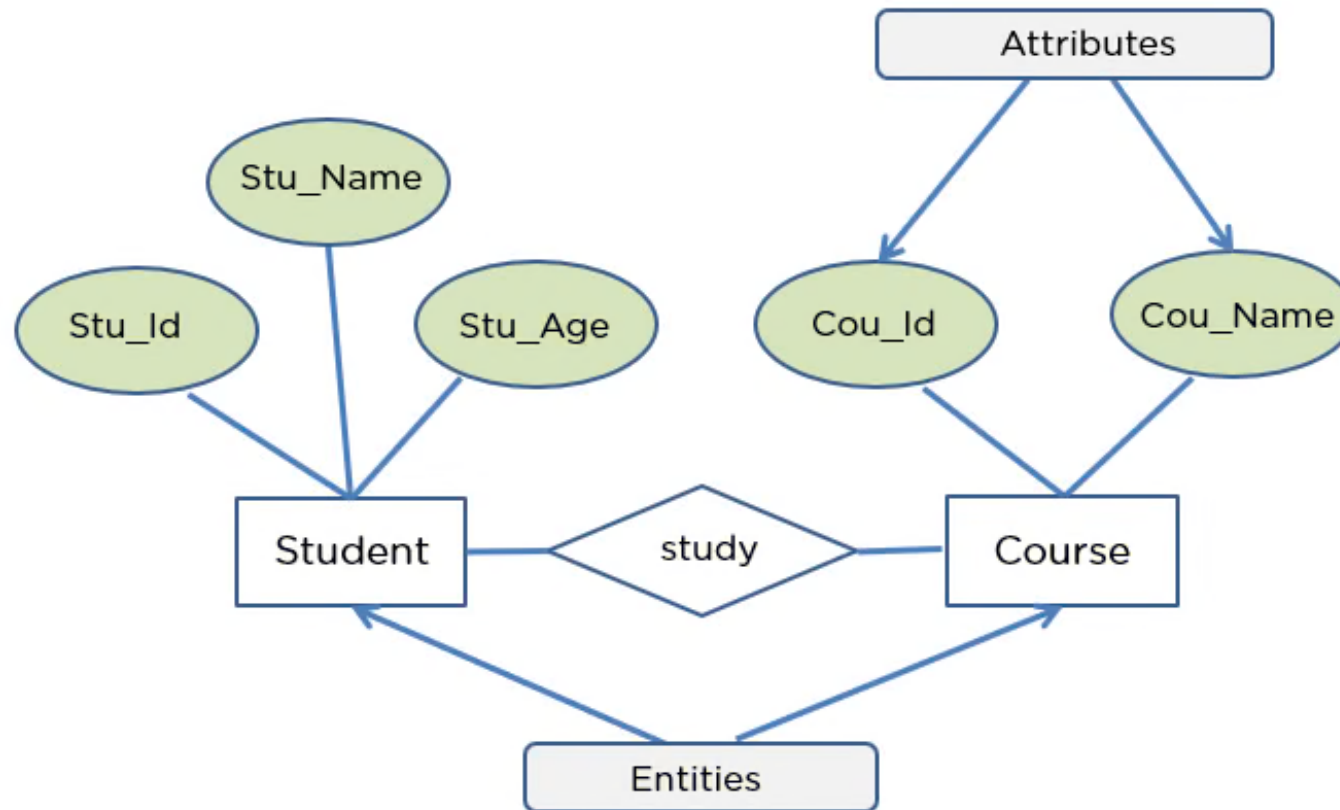
ellipses: represent attributes

diamonds: represent relationships among entity sets

lines: link attributes to entity sets and entity sets to relationships

Data Models

- Beside entities and relationship among them, E-R model has a capability to enforce constraints, mapping cardinalities which tell no. of entities to which another entity can be associated via relationship set.



Data Models

c. The Relational Model

- Relational model describes database design by a collection of tables (relations).
- It represents both data and their relationships among those data.
- Each table consist number of columns (attributes) with unique names.
- It is a most widely used data model.
- Relational model is lower level abstraction than E-R model. Database model are often carried out in E-R model and then translated into relational mode.

Example:

customer_id	customer_name	customer_city
C01	X	A
C02	Y	B
C03	Z	A
C04	X	A

(a) Customer relation

account_number	balance
A1	200
A2	300
A3	500
A4	500

(b) Account relation

customer_id	account_number
C01	A1
C02	A2
C03	A3

(c) Depositor relation

Data Models

b. Object oriented model

- Object oriented data model is extension to E-R model with the notion of, methods (functions), encapsulation inheritance and object identity. It is based on collection of objects, like the E-R model. An object contains values stored in instance variables within the object.
- Example:

Consider an object representing a bank account.

- The object may contain instance variables `account_number` and `balance`.
- The object may contain a method `pay-interest` which adds interest to the balance.

Example:

ObjectDB is a high-performance, pure object-oriented database for Java and .NET.

PostgreSQL is the most successful example of a "hybrid" system. It is a relational database (like MySQL) but includes native object-oriented features like user-defined types, table inheritance, and complex nesting.

d. Hierarchical Model

- Data is organized into a **tree-like structure**.
- Each child record has only one parent, but a parent can have multiple children.
- This was common in early mainframe systems like **IBM's IMS** (world's most famous and widely used hierarchical database)
 - **Structure:** 1:N (One-to-Many) relationships.
 - **Constraint:** Deleting a parent automatically deletes all its children.

Example:

Your computer's file system (NTFS for Windows) is the most basic example of hierarchical software.

Root: The drive (C:).

Parent: Folders.

Child: Sub-folders or Files.

Data Models

e. The Network Model

- An extension of the hierarchical model, but it allows a child record to have multiple parents. This creates a **graph-like structure** rather than a strict tree.
 - **Structure:** M:N (Many-to-Many) relationships.
 - **Complexity:** Harder to design and maintain compared to the relational model.
- **Example:** Neo4j or Amazon Neptune.
- **Application:** Social media (Facebook's "Social Graph"), recommendation engines (Netflix/Amazon)

Instances and Schemas

Schema:

- The overall design of the database which is not expected to change frequently is called the database schema.
- The **Schema** is the skeleton or the logical structure of the entire database.
- It defines the "blueprint"—what tables exist, what columns they have, and how those tables relate to one another.

Categories of Schema:

- **Physical Schema:** Describes how data is stored on disk.
- **Logical Schema:** Describes the data in terms of tables, views, and integrity constraints.

Instance:

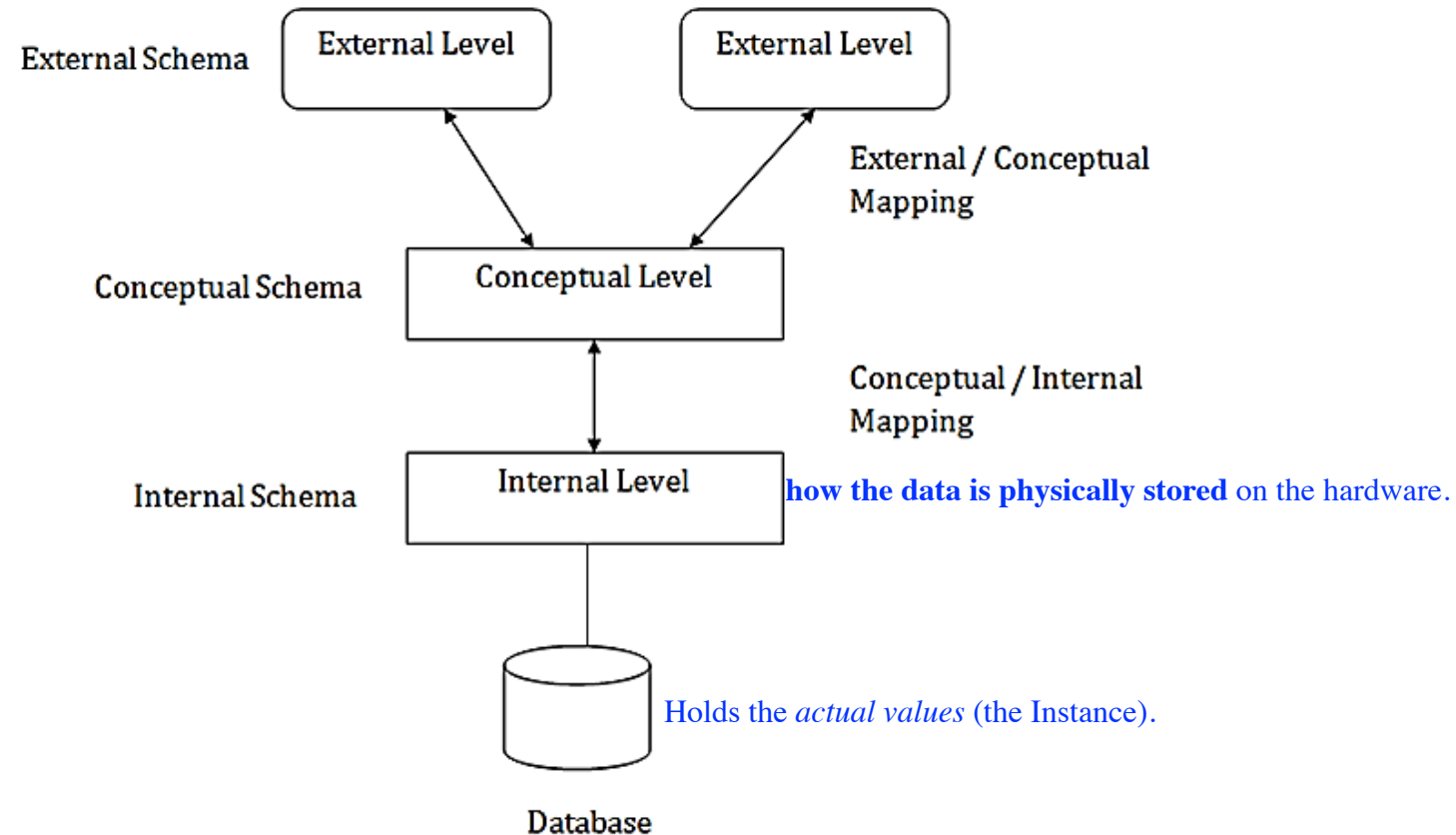
- A **Database Instance** is the actual data stored in the database at a specific moment in time. It is also known as database state.
- An instance is created and updated as users insert, delete, or modify records.
- It changes constantly. Example: *Every time you add a new student to a "Student" table, you have created a new instance of the database.*

- Database Schema Vs Database Instance

Feature	Database Schema	Database Instance
Definition	The overall design/blueprint of the DB.	The collection of information at a given moment.
Stability	Relatively constant; changes are infrequent.	Changes frequently as data is modified.
Analogy	A class or data type in C++/Python.	An object or a variable's current value.
Example	Student(ID: int, Name: string, Age: int)	(101, "Alice", 22), (102, "Bob", 21)

Three Schema Architecture/ANSI/SPARC Architecture

- This framework is used to describe the structure of a specific database system.
- The three schema architecture is also used to separate the user applications and physical database.
- The three schema architecture contains three-levels. It breaks the database down into three different categories.



Internal Level

- The internal level has an internal schema which describes the physical storage structure of the database.
- The internal schema is also known as a physical schema.
- It uses the physical data model. The physical level is used to describe complex low-level data structures in detail.

Conceptual Level

- The conceptual schema describes the design of a database at the conceptual level. Conceptual level is also known as logical level.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.
- Programmers and database administrators work at this level.

External level

- At the external level, a database contains several schemas that sometimes called as subschema. The subschema is used to describe the different view of the database.
- An external schema is also known as view schema.
- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.

- **External Schema (Presentation Tier):**

- The presentation tier is the user interface or client layer of the application.
- It is responsible for presenting data to the user and receiving input from the user.
- This tier communicates with the Application Tier to process user requests and display relevant information.
- This tier can be a web browser, mobile app, or desktop application.
- It ensures the application is user-friendly.

Example: If you're using a banking app, the presentation tier would display your account balance, allow you to make transfers, and display results based on your actions.

- **Application Tier (Business Logic Layer/Back-End)**

- It acts as the intermediary between the Presentation Tier and the Data Management Tier.
- It is responsible for processing, performing calculations and managing the business logic of the application.
- *This tier communicates with the presentation tier to receive user input and communicates with the data management tier to retrieve or store data.*
- Programmers and database administrators work at this level.
- Examples: Django, Node.js, and Java

- **Data Tier (Database/Storage):**
- The bottom layer that stores and manages data, such as MySQL, PostgreSQL, or Oracle databases, ensuring data persistence.

Mapping in Three Schema Architecture

Mapping is the process that connects the different levels of the database.

It acts as a translator that allows data to be transformed as it moves between the user's view and the physical storage.

- **External / Conceptual Mapping:**

- It defines how a specific user's view is derived from the main database. For example, it might map a field called Birthday in a user's app to a field called Date_of_Birth in the main database.
- It provides Logical Data Independence.
- If the database administrator adds a new table or changes a relationship in the Conceptual Schema, they only need to update this mapping (*The DBA modifies the SQL code of the External View, update metadata*). The user's application remains "unaware" of the change and continues to work normally.

- **Conceptual/ Internal Mapping:**

- It describes how logical records and fields are physically represented on the disk. It covers things like file organizations, indexing, and data compression
- It provides Physical Data Independence.
- If the database administrator decides to change the storage from a B-Tree index to a Hash index, or moves the data from one physical drive to another (cloud), the **Conceptual/Internal Mapping** is updated—> The DBA issues a command to the DBMS. Instead of the mapping saying "Table X is in C:/Local/Data," it is updated to say "Table X is at cloud://storage/data."
- Because of this, the **Conceptual Schema** remains unchanged, meaning the users never even know the physical storage moved.

Data Independence

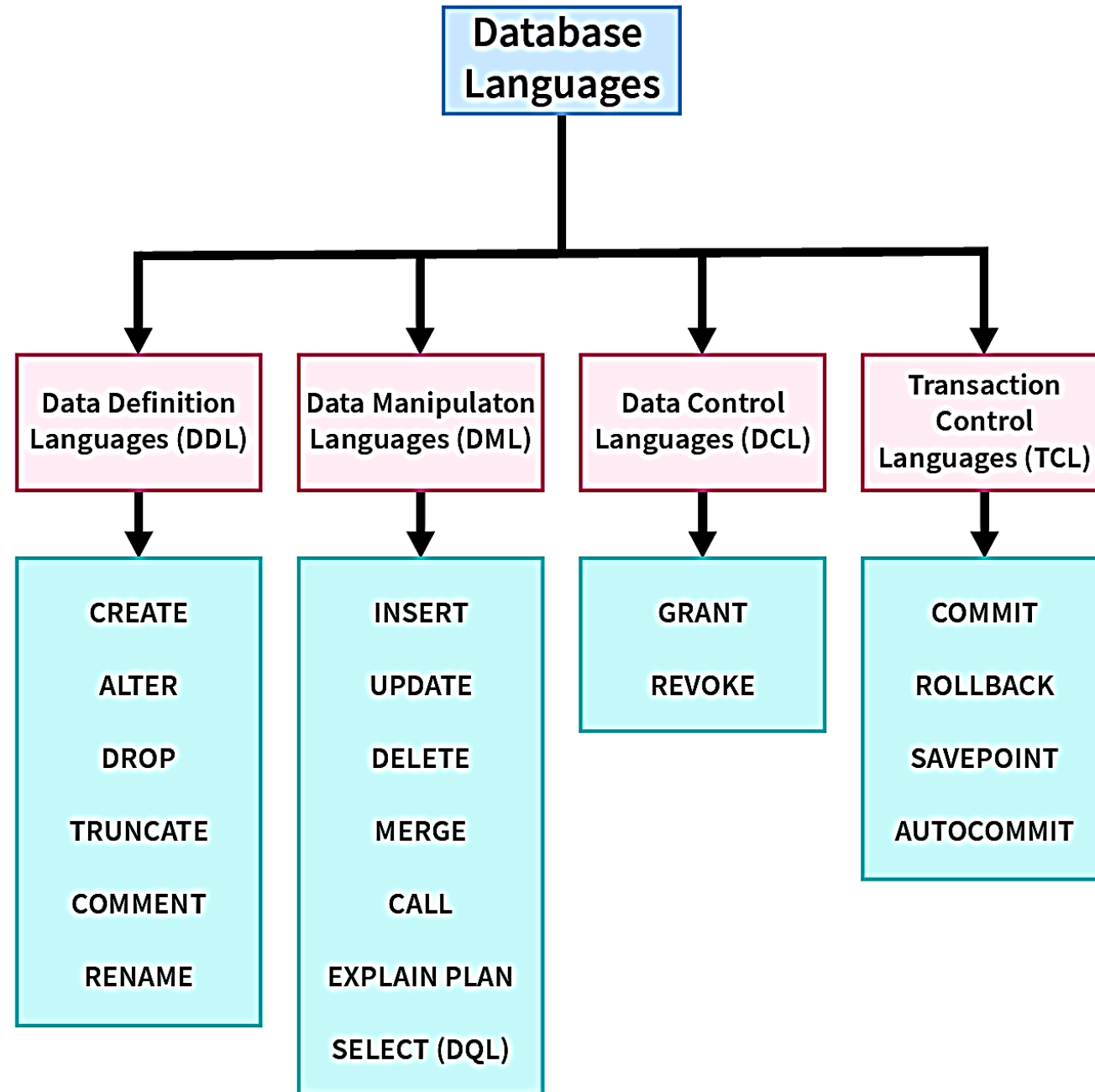
- Data independence is an ability to modify a schema definition in one level without affecting scheme definition in higher level.
- There are two types of data independence.

Physical data independence

- It is an ability to modify the physical scheme without causing application programs to be rewritten.
- Modification at this level usually required for performance improvement reason.

Logical data independence

- It is an ability to modify the conceptual/logical scheme without causing application programs to be rewritten.
- Logical scheme needs to modify if we required to modify logical structure of database.
- Logical data independence is harder to achieve since application programs are usually dependent on logical structure of the data.



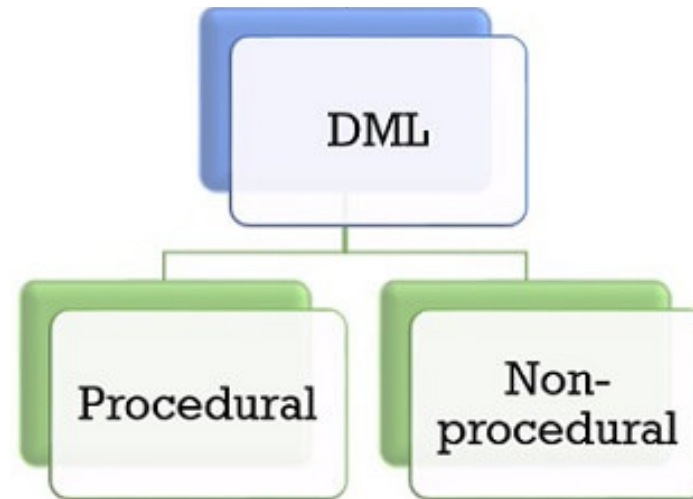
Data Definition Language

- DDL stands for Data Definition Language. It is used to define database structure.
- It is used to create schema, tables, indexes, constraints, etc. in the database.
- Using the DDL statements, you can create the skeleton of the database.
- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.
- Here are some tasks that come under DDL:
- **Create:** It is used to create objects in the database.
- **Alter:** It is used to alter the structure of the database.
- **Drop:** It is used to delete objects from the database.
- **Truncate:** It is used to remove all records from a table.
- **Rename:** It is used to rename an object.
- **Comment:** It is used to comment on the data dictionary.
- These commands are used to update the database schema that's why they come under Data definition language.

Data Manipulation Language

- **DML** stands for **Data Manipulation Language**. *It is used for accessing and manipulating data in a database. It handles user requests.*
- Here are some tasks that come under DML:
- **Select:** It is used to retrieve data from a database.
- **Insert:** It is used to insert data into a table.
- **Update:** It is used to update existing data within a table.
- **Delete:** It is used to delete all records from a table.
- **Merge:** It performs UPSERT operation, i.e., insert or update operations.
- **Call:** It is used to call a stored procedure.
- **Explain Plan:** It has the parameter of explaining data. It reveals the "execution path" the database will take. It shows whether the system will perform a "Full Table Scan" (reading everything) or use an "Index Scan"
- **Lock Table:** It controls concurrency.

- There are two types of DML
 - a. **Procedural DML:** user required to specify what data are needed and how they get those data.
Example: Stored Procedures, Triggers
 - b. **Nonprocedural (Declarative) DML:** user only required to what data needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than procedural DMLs. However, since a user does not have to specify how to get data, the database system has to figure out an efficient means of accessing data.
- The DML component of SQL is nonprocedural.



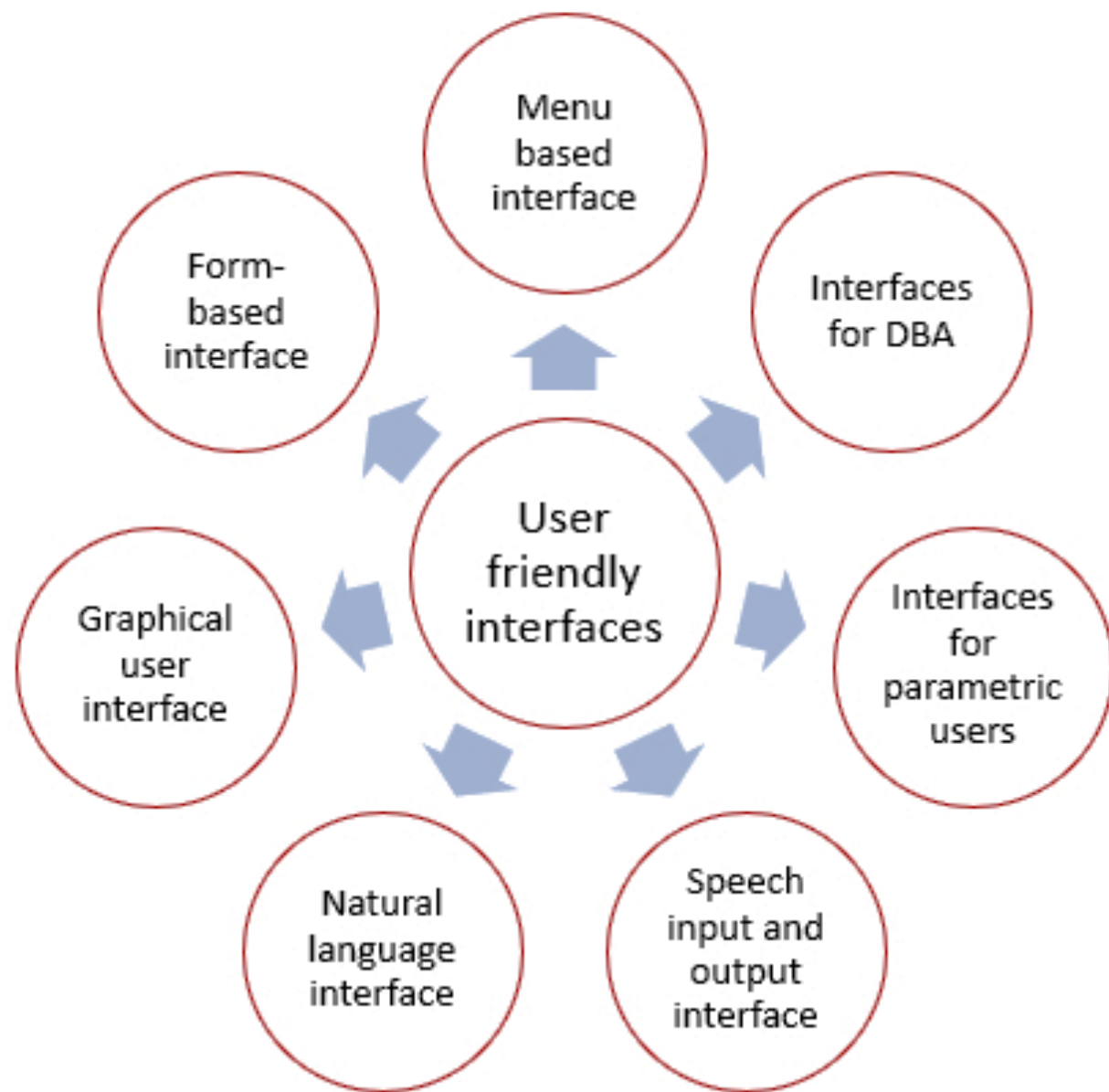
Data Control Language (DCL)

- DCL is the component of SQL used to control access to the data stored within the database. *While DDL defines the structure and DML manages the content, DCL acts as the security layer of the architecture.*
- **Grant:** It is used to give user access privileges to a database.
- **Revoke:** It is used to take back permissions from the user.

Transaction Control Language(TCL)

- TCL is the set of commands used to manage transactions within a database.
- A **transaction** is a logical unit of work that consists of one or more DML statements (like INSERT, UPDATE OR DELETE), that must either all succeed or all fail together.
- Here are some Commands that come under TCL:
- **Commit:** It is used to save the transaction on the database.
- **Rollback:** It is used to restore the database to original since the last Commit.
- **Savepoint:** It allows you to roll back a portion of the transaction without undoing everything.

Database User Interfaces



Menu-based interfaces*(For Browsing)*

- Present a list of options (menus) to the user.
- These are common in web applications or ATM machines.
- The user doesn't need to know SQL; they just select an action, and the interface triggers the underlying DML.

Forms-based interfaces *(For Data Entry)*

- These types of interfaces display a form to user. The user is able to fill the entries to insert data.
- These forms are usually designed and programmed by naïve users as interfaces for recorded transactions.
- *Example create accounts on a website, or a student registering for classes*

Graphical user interfaces

- Tools like phpMyAdmin, MySQL Workbench.
- These often show the database structure as a schema diagram, allowing users to click and drag to build queries.

Natural language interfaces

- Allow users to submit requests written in English or other human languages and attempt to understand them. A natural language interface usually has its own schema, which is similar to the database conceptual schema.
 - User-friendly for non-technical users.
 - Limited capabilities and may require clarification dialogues for ambiguous queries.
 - Example: Modern Text-to-SQL tools.

Speech input and output interfaces

- Speech-based interfaces allow users to speak queries and receive results verbally. Applications with limited vocabulary, such as flight information, bank accounts or telephone directories.
- Enables access for users who cannot use keyboards.
- Speech input is converted into parameters for queries; output is converted from text/numbers into speech.
- Example: Google Assistant/Siri

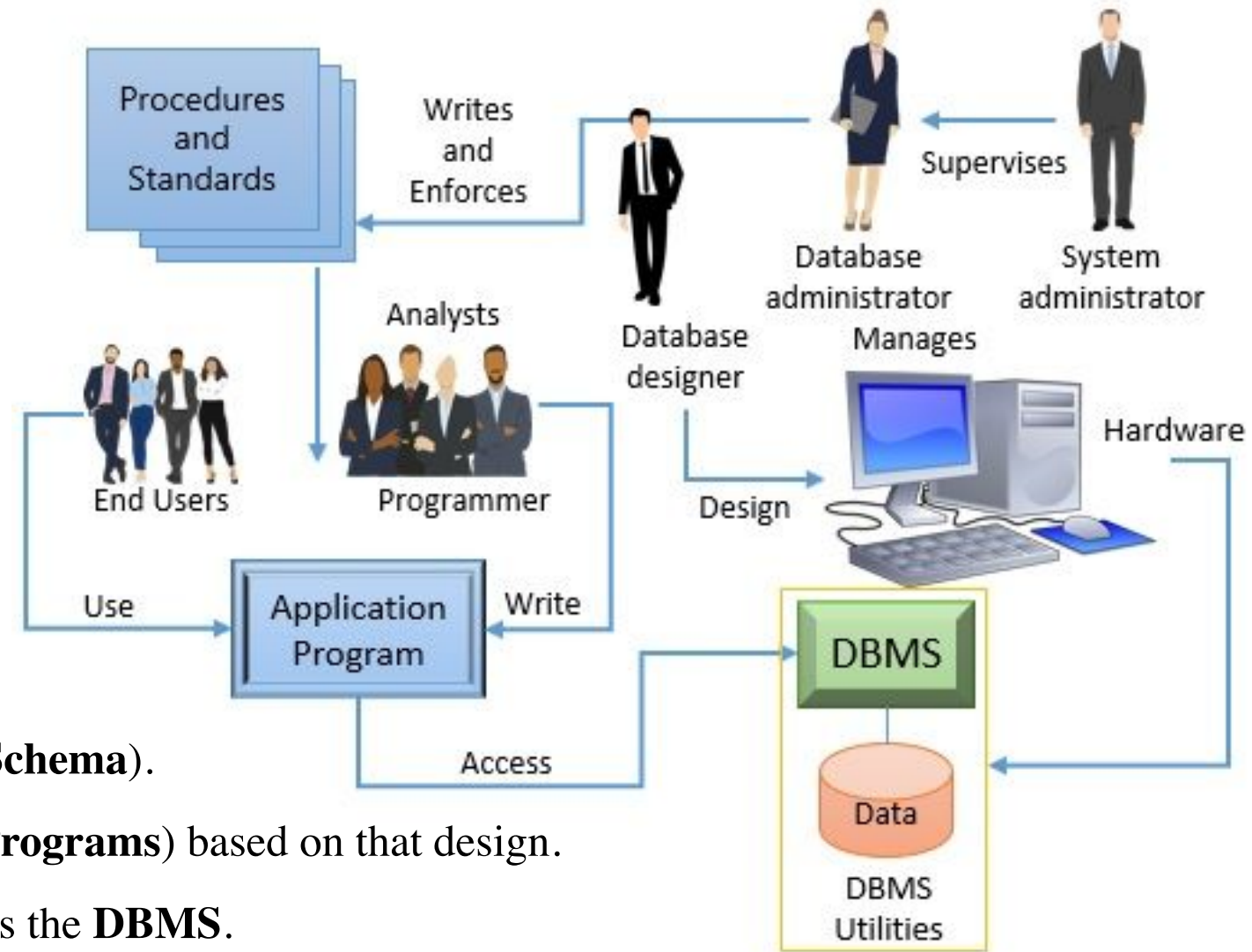
Interfaces for parametric users

- Parametric users like bank tellers have a small set of operations that they must perform repeatedly.
- These interfaces contain some commands to perform a request with minimum keystrokes.
- Example: Bank teller systems, ATM interfaces.

Interfaces for DBA

- These interfaces contain some commands for creating accounts, to manipulate the database to perform some operations on the database.
- These interfaces are specially used by the database administrators.

Database System Environment



Summary of the Flow

1. The **Designers** create the blueprint (**Conceptual Schema**).
2. The **Programmers** build the tools (**Application Programs**) based on that design.
3. The **DBA** sets the rules (**Procedures**) and manages the **DBMS**.
4. The **End Users** interact with the **Application Program**,
which communicates with the **DBMS**, which finally fetches the **Data** from the **Hardware**.

Database System Environment

- The **database system environment** is comprised of the components that are meant for defining and managing the data that we collect, store, manage and use in the database environment.

1. Hardware

- The hardware component of the database system environment includes all the physical devices that comprise the database system. It includes storage devices, processors, input and output devices, printers, network devices and many more.

2. Software

- The software component of the database environment includes all the software that we require to access, store and regulate the database. Like operating systems, DBMS and application programs and utilities. The operating system invokes computer hardware, and let other software runs. DBMS software controls and regulates the database. The application program and utilities access the database and if required you can even manipulate the database.

3. People

- If talk of the people component then it will include all the people who are related to the database. There may be a group of people who will access the database just to resolve their queries i.e. end-user, there may be people that are involved in designing the database i.e. database designer.
- Some may be involved in designing the applications that will have an interface through which data entry is possible i.e. database programmer and analyst and some may also be there to monitor the database i.e. database administrator.

4. Procedures

- The procedure component of the database environment is nothing but the function that regulates and controls the use of the database.

5. Data

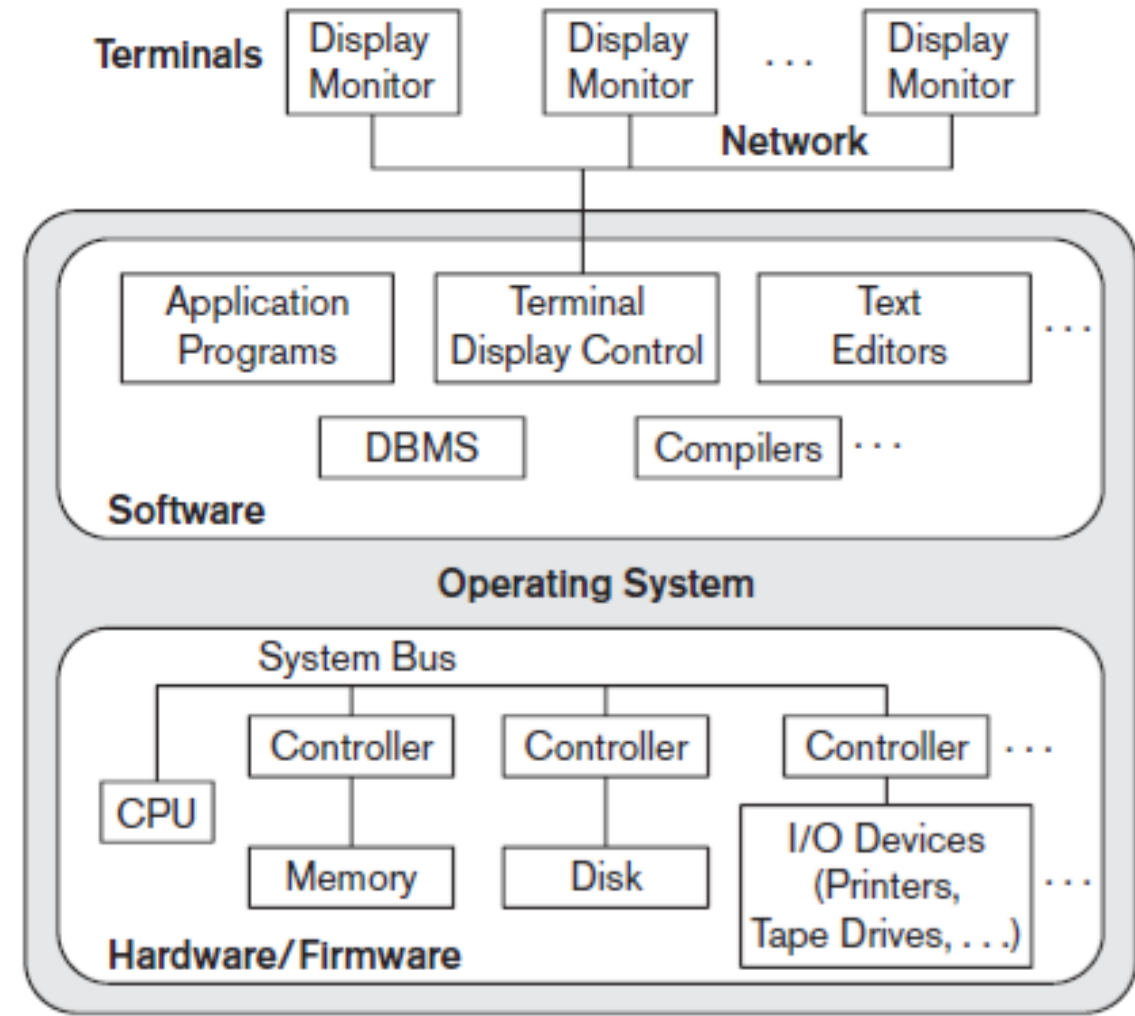
- Data component include a collection of related data which are the known fact that can be recorded and it has an implicit meaning in the database.

System Utilities

- Database system utilities are the tools that can be used by the database system administrator to control and manage the database system.

Centralized Architecture for DBMS

- In Centralized Architecture, the mainframe computers are used for processing all system functions including User application Programs and User Interface Programs as well as DBMS functionalities.
- This is because in earlier days, most users accessed such systems via Computer Terminals, which can't Process and they have only display capability.
- As the days pass by, we are now having Personal Computers(PC's) in the market. But still in the beginning Centralized Architecture for DBMS was used.
- Gradually the DBMS systems started to make use of Processing Power in the used side i.e Computers have come with Processing Power and in turn led to the use of Client/Server Architecture.



Client/Server Architecture for DBMS

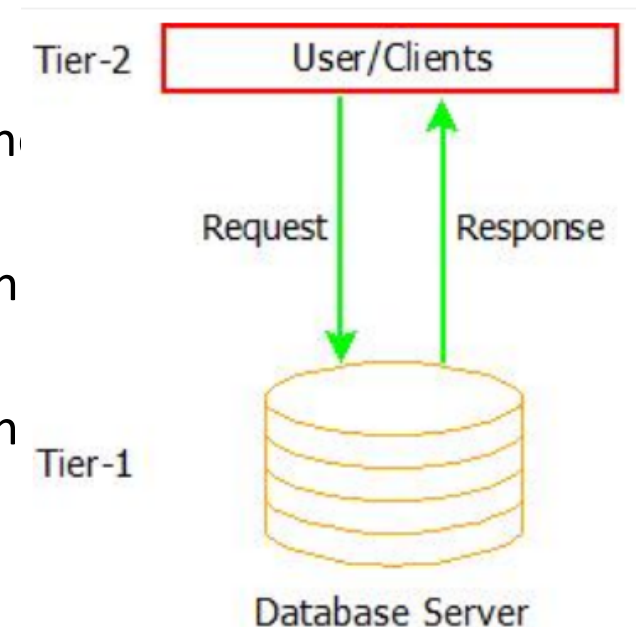
Client/Server Architecture:

1)1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

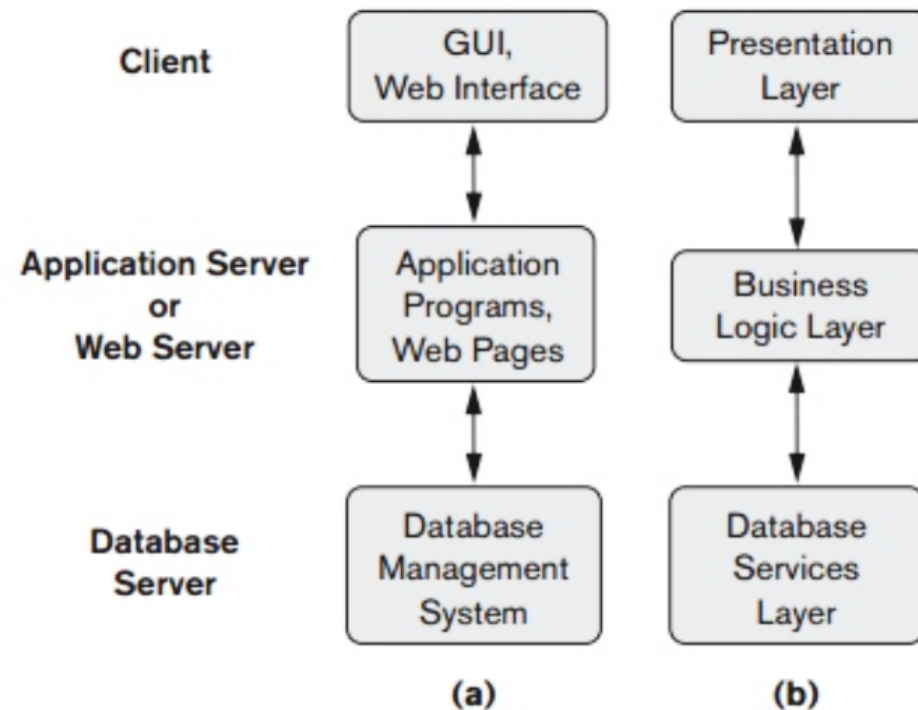
2)Two Tier Client/Server Architecture for DBMS:

- Here Two-tier means that our Architecture has two layers, which are client layer and Data layer.
- In Client layer we have several Client machines which can have the access to the database server.
- The query from client will be processed by the Database server and in turn the queried information will be sent to the client machine.



3) Three-Tier client/server Architecture for DBMS:

- There is an additional layer which acts as an intermediate between Client layer and Data layer called Business logic layer.
- Business logic layer is the layer where the Application Programs are processed. Here the Application Programs are processed in the Application server itself, which makes it different from Two-tier Architecture where queries are processed in the database server.
- Simply the Client machines will contact Application Server which in turn processes our Application Programs and fetches the Required Data from Database and then sends this Information back to the client machine in the suitable format only.



- Database management systems can be classified based on several criteria, such as the data model, user numbers and database distribution.

Classification Based on Data Model

- The most popular data model in use today is the **relational data model**. Well-known DBMSs like Oracle, MS SQL Server, DB2 and MySQL support this model. Other traditional models, such as **hierarchical data models** and **network data models**, are still used in industry mainly on mainframe platforms. However, they are not commonly used due to their complexity. These are all referred to as *traditional models* because they preceded the relational model.
- In recent years, the newer ***object-oriented data models*** were introduced. This model is a database management system in which information is represented in the form of objects as used in object-oriented programming. Object-oriented databases are different from relational databases, which are table-oriented. Object-oriented database management systems (OODBMS) combine database capabilities with object-oriented programming language capabilities.

Classification Based on User Numbers

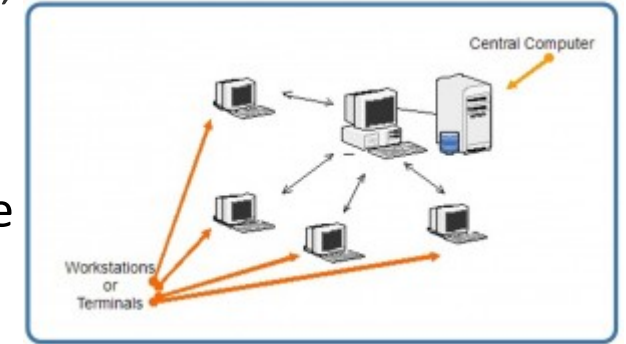
- A DBMS can be classification based on the number of users it supports. It can be a ***single-user database system***, which supports one user at a time, or a ***multiuser database system***, which supports multiple users concurrently.

Classification Based on Database Distribution

- There are four main distribution systems for database systems and these, in turn, can be used to classify the DBMS.

1. Centralized systems

- With a ***centralized database system***, the DBMS and database are stored at a single site that is used by several other systems too.



2. Distributed database system

- In a ***distributed database system***, the actual database and the DBMS software are distributed from various sites that are connected by a computer network

a. Homogeneous distributed database systems

- Homogeneous distributed database systems* use the same DBMS software from multiple sites. Data exchange between these various sites can be handled easily.

b. Heterogeneous distributed database systems

- In a *heterogeneous distributed database system*, different sites might use different DBMS software, but there is additional common software to support data exchange between these sites.

